Design and Verification of a Single-Cycle RISC-V Processor

Imad Uddin
Department of Electrical Engineering
Ghulam Ishaq Khan Institute (GIKI)
Chip Design and Verification
Email: [imaduddin182@gmail.com]

***Abstract*—This report presents the design, implementation, and simulation of a 32-bit Single-Cycle RISC-V Processor developed in Verilog HDL and verified using Xilinx Vivado. The processor supports R-Type, I-Type, S-Type, B-Type, and J-Type instructions of the RV32I instruction set. A complete datapath and control architecture were implemented. Simulation results demonstrate correct execution of arithmetic, logical, memory access, branch, and jump instructions.**

***Index Terms*—RISC-V, Single-Cycle Processor, RV32I, Verilog HDL, Vivado, Computer Architecture, FPGA Design.**

I. Introduction

RISC-V is an open-source Instruction Set Architecture (ISA) developed to provide a flexible and extensible processor platform for both academic and industrial applications. RISC-V allows researchers and developers to design custom processors without licensing restrictions. Its modular design, simplicity, and scalability have contributed to its rapid adoption in embedded systems and computer architecture research.

The Reduced Instruction Set Computer (RISC) emphasizes the use of a small set of simple instructions that can be executed efficiently by hardware. This approach reduces processor complexity and improves design efficiency. Among various processor organizations, the single-cycle processor is one of the simplest implementations because every instruction completes all stages of execution within a single clock cycle. These stages include instruction fetch, instruction decode, execution, memory access, and write-back.

Although single-cycle processors are not optimized for high performance due to their long clock period, they provide an excellent educational platform for understanding processor architecture and datapath design. The architecture clearly demonstrates the interaction between the control unit, arithmetic logic unit, register file, and memory subsystems.

In this work, a 32-bit RV32I Single-Cycle RISC-V processor was designed and implemented using Verilog HDL. The processor supports arithmetic, logical, memory access, branch, and jump instructions. The design was simulated and verified using Xilinx Vivado to ensure correct functionality of all supported instruction formats.

II. Objectives

The objectives of this design are:

- Design a 32-bit Single-Cycle RISC-V Processor.
- Implement the processor using Verilog HDL.
- Support R-Type, I-Type, S-Type, B-Type, and J-Type instructions.
- Verify functionality through Vivado simulation.
- Analyze datapath operation for each instruction format.

III. Processor Architecture

The processor consists of the following hardware modules:

- Program Counter (PC)
- Instruction Memory
- Register File
- Immediate Generator
- Arithmetic Logic Unit (ALU)
- Data Memory
- Control Unit
- ALU Control Unit
- Multiplexers
- Branch and Jump Logic

The proposed processor follows a standard single-cycle architecture in which every instruction is executed within one clock cycle. The processor datapath consists of several interconnected hardware modules responsible for instruction fetching, decoding, execution, memory access, and result storage.

The Program Counter (PC) maintains the address of the current instruction being executed. During normal operation, the PC increments by four bytes to access the next instruction. For branch and jump instructions, the PC is updated with a target address calculated by dedicated control logic.

Instruction Memory stores the machine code program and provides a 32-bit instruction corresponding to the address supplied by the Program Counter. The fetched instruction is then decoded to determine the required operation and associated control signals.

The Register File contains thirty-two 32-bit general-purpose registers defined by the RV32I specification. Two source operands can be read simultaneously, while a third register can be updated during the write-back stage. This structure enables efficient execution of arithmetic and logical operations.

The Arithmetic Logic Unit (ALU) performs arithmetic and logical computations. Supported operations include addition, subtraction, bitwise AND, OR, XOR, and comparison functions. The ALU receives control signals from the ALU Control

Unit, which determines the operation based on instruction fields.

Immediate Generator extracts immediate values from different instruction formats and performs sign extension when necessary. These immediate values are used for arithmetic instructions, memory addressing, branching, and jumping operations.

Data Memory supports load and store instructions. For load operations, data is retrieved from memory and written back into the register file. For store operations, register contents are written into memory at the address computed by the ALU.

The Control Unit and **ALU Control Unit** generate all necessary control signals required for datapath operation. These signals determine register writes, memory accesses, ALU operations, branching decisions, and result selection.

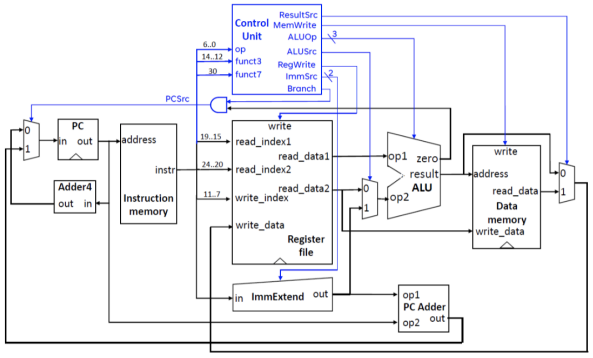

Fig. 1. Single Cycle RISC-V uProcessor Architecture

IV. Instruction Formats

The RV32I instruction set architecture defines multiple instruction formats to support different classes of operations while maintaining a fixed instruction length of 32 bits. Each format contains specific fields such as opcode, source registers, destination registers, function codes, and immediate values. The control unit decodes these fields to generate the appropriate control signals required for instruction execution. The processor implemented in this work supports R-Type, I-Type, S-Type, B-Type, and J-Type instruction formats.

A. *R-Type Instructions*

R-Type instructions are used for register-to-register arithmetic and logical operations. These instructions utilize two source registers, denoted as *rs1* and *rs2*, and store the computation result in the destination register *rd*. The operation performed by the Arithmetic Logic Unit (ALU) is determined by the *funct3* and *funct7* fields in combination with the opcode.

In the proposed processor, R-Type instructions are used to perform fundamental arithmetic and logical functions such as addition, subtraction, bitwise AND, bitwise OR, and bitwise XOR. During execution, both operands are read from the register file, processed by the ALU, and the resulting value is written back to the destination register. Since all operands are stored in registers, no memory access is required during execution. Examples of supported R-Type instructions

```
ADD, SUB, AND, OR, XOR                    (1)
```

B. *I-Type Instructions*

I-Type instructions are designed for operations that require an immediate constant value. Instead of obtaining both operands from registers, one operand is read from a register while the second operand is provided by a 12-bit immediate field embedded within the instruction. The immediate value is sign-extended to 32 bits before being used by the ALU.

This instruction format is commonly used for arithmetic operations involving constants as well as memory load operations. For arithmetic instructions such as ADDI, the ALU adds the sign-extended immediate value to the register operand and writes the result back to the destination register. For load instructions such as LW, the ALU calculates an effective memory address using the base register and immediate offset. The data stored at the computed address is then read from memory and written back into the destination register. Used for immediate arithmetic and load operations.

Examples of supported I-Type instructions

```
ADDI, LW                                  (2)
```

C. *S-Type Instructions*

S-Type instructions are used for memory store operations. In this format, the immediate value is divided into two separate fields within the instruction and later reconstructed by the Immediate Generator. The instruction contains two source registers: one register provides the base address while the other provides the data to be written into memory.

During execution, the ALU computes the effective memory address by adding the sign-extended immediate value to the contents of the base register. The data stored in the second source register is then transferred to the Data Memory module and written at the calculated address. Since the objective of the instruction is to store data into memory, no register write-back operation occurs. Used for store instructions.

```
SW                                        (3)
```

D. *B-Type Instructions*

B-Type instructions implement conditional branch operations and are essential for controlling program flow. Similar to S-Type instructions, the immediate value is distributed across multiple instruction fields and reconstructed by the Immediate Generator. The branch target address is computed by adding the sign-extended branch offset to the current Program Counter value.

During execution, the ALU compares the contents of two source registers and generates a condition flag. The branch control logic evaluates this flag together with the instruction type to determine whether the branch condition has been satisfied. If the condition evaluates to true, the Program Counter is updated with the branch target address; otherwise, sequential

execution continues by incrementing the Program Counter by four bytes. Examples of supported B-Type instructions include

`BEQ, BNE` (4)

E. *J-Type Instructions*

J-Type instructions are used for unconditional jump operations that modify program execution flow. The instruction contains a large immediate field capable of representing relatively long jump distances. The immediate value is reconstructed and sign-extended by the Immediate Generator before being used to calculate the jump target address.

In the implemented processor, the JAL (Jump and Link) instruction is supported. During execution, the processor calculates the jump target address and updates the Program Counter accordingly. Simultaneously, the return address, represented by the value PC+4, is written into the destination register. This mechanism enables subroutine calls and returns, which are fundamental for structured program execution.

The J-Type format therefore provides an efficient method for implementing function calls and unconditional control transfers within the processor. Used for jump operations.

`JAL` (5)

V. Datapath Analysis

The datapath represents the flow of data through the processor during instruction execution. In a single-cycle processor, all stages of instruction processing, including instruction fetch, decode, execute, memory access, and write-back, are completed within a single clock cycle. The datapath integrates the Program Counter, Instruction Memory, Register File, Immediate Generator, Arithmetic Logic Unit (ALU), Data Memory, and multiplexers under the control of the Control Unit. Different instruction formats utilize different portions of the datapath depending on the required operation.

A. *R-Type Datapath*

R-Type instructions perform register-to-register arithmetic and logical operations. The execution begins when the Program Counter supplies the instruction address to the Instruction Memory. The fetched instruction is decoded, and the source register addresses *rs1* and *rs2* are sent to the Register File.

The Register File simultaneously reads the contents of both source registers and forwards them to the ALU inputs. Based on the opcode, funct3, and funct7 fields, the ALU Control Unit selects the appropriate ALU operation such as addition, subtraction, AND, OR, or XOR. The ALU then performs the requested operation and produces the result.

Since R-Type instructions do not require memory access, the ALU output is directly routed through the write-back multiplexer and stored in the destination register *rd*. The Program Counter is incremented by four to fetch the next sequential instruction. This execution path provides efficient register-based computation with minimal datapath complexity.

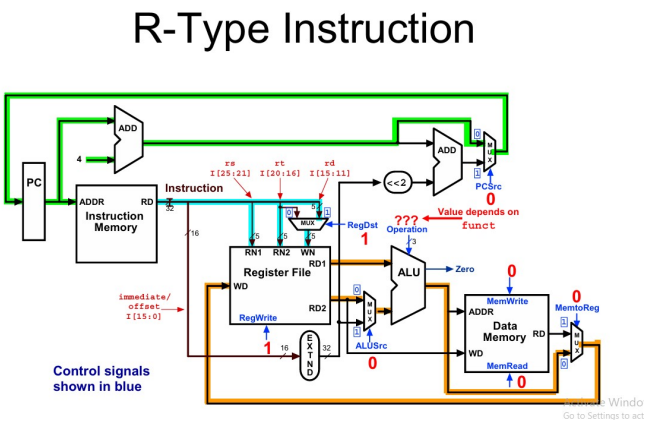

Fig. 2. Datapath for R-Type Instructions

B. *I-Type Datapath*

I-Type instructions utilize an immediate operand embedded within the instruction. After instruction fetch and decode, the Register File reads the source register specified by *rs1*. Simultaneously, the Immediate Generator extracts the 12-bit immediate field and sign-extends it to 32 bits.

A multiplexer controlled by the ALUSrc signal selects the immediate value as the second ALU operand instead of a register value. The ALU then performs the specified arithmetic operation. For instructions such as ADDI, the ALU result is forwarded directly to the write-back stage and stored in the destination register.

For load instructions such as LW, the ALU computes the effective memory address by adding the immediate offset to the base register value. The calculated address is used to access Data Memory. The retrieved memory data is then selected by the ResultSrc multiplexer and written into the destination register. This datapath enables both immediate arithmetic operations and memory loading functionality.

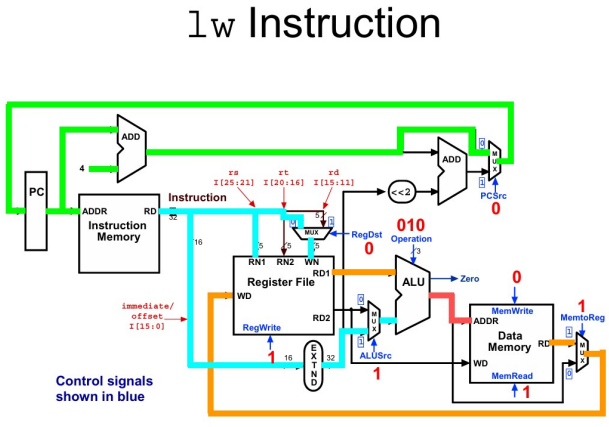

Fig. 3. Datapath for I-Type Instructions

C. *S-Type Datapath*

S-Type instructions are responsible for storing data into memory. Following instruction decoding, the Register File reads two source registers. The first register provides the base

address while the second register contains the data that will be written into memory.

The Immediate Generator reconstructs the split immediate field and performs sign extension. The ALU calculates the effective memory address by adding the immediate offset to the base register value. The resulting address is forwarded to the Data Memory module.

The contents of the second source register are supplied to the write-data port of Data Memory. When the MemWrite control signal is asserted, the data is stored at the computed memory location. Since store instructions do not generate a result that must be written back into the register file, the RegWrite signal remains disabled during execution.

`sw` Instruction

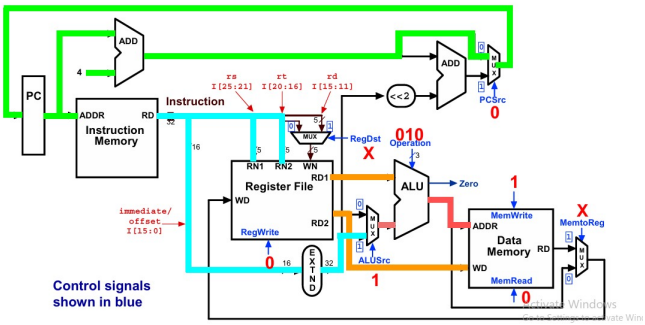

Fig. 4. Datapath for S-Type Instructions

D. B-Type Datapath

B-Type instructions implement conditional branching and modify program flow based on comparison results. During execution, the Register File provides the contents of the two source registers to the ALU. The Immediate Generator reconstructs the branch offset from the instruction fields and performs sign extension.

The ALU compares the source register values, typically using subtraction, and generates a Zero flag. The Control Unit combines the branch control signal with the Zero flag to determine whether the branch condition has been satisfied.

Simultaneously, a branch target address is calculated by adding the sign-extended branch offset to the current Program Counter value. If the branch condition evaluates to true, the Program Counter is updated with the branch target address. Otherwise, the Program Counter advances normally to the next sequential instruction address. This datapath enables conditional program execution and decision-making operations.

`beq` Instruction

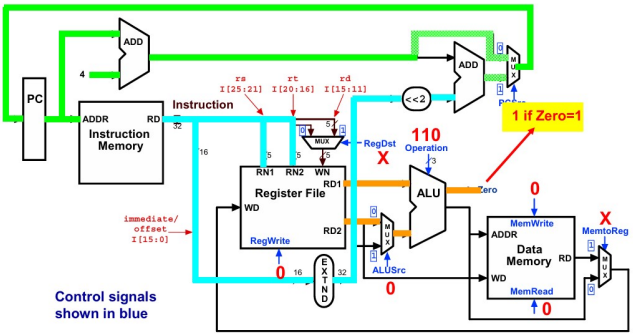

Fig. 5. Datapath for B-Type Instructions

E. J-Type Datapath

J-Type instructions provide unconditional jump functionality. After instruction decoding, the Immediate Generator extracts and reconstructs the jump offset encoded within the instruction. The jump target address is calculated by adding the sign-extended offset to the current Program Counter value.

For the JAL instruction, the processor simultaneously computes the return address, which corresponds to the value of PC+4. This return address is routed through the write-back path and stored in the destination register specified by *rd*. At the same time, the Program Counter is updated with the jump target address, causing execution to continue from the new location. The complete datapath integrates all functional units under the supervision of the Control Unit and ALU Control Unit. Depending on the instruction opcode, different control signals are generated to select ALU inputs, enable register writing, access memory, and determine Program Counter updates. Because the processor follows a single-cycle architecture, every instruction traverses the required datapath components and completes execution within one clock period.

Datapath Executing `j`

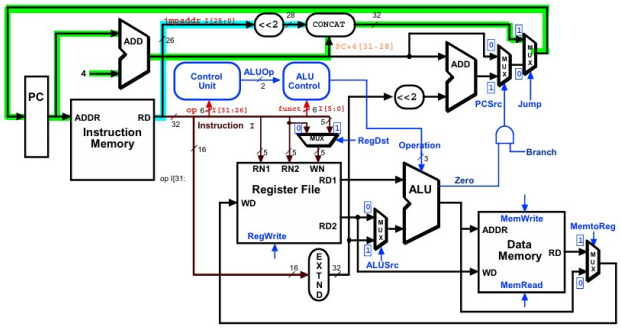

Fig. 6. Datapath for J-Type Instructions

Although this approach simplifies processor design and verification, the clock period must be long enough to accommodate the worst-case instruction path, typically involving memory access operations. Nevertheless, the single-cycle datapath provides a clear and effective framework for understanding processor operation and serves as a foundation for more advanced pipelined architectures.

VI. Control Unit Design

The control unit is a key component of the single-cycle RISC-V processor. It is responsible for decoding the instruction opcode and generating the required control signals to manage the datapath during execution. Since all instructions execute in one clock cycle, the control unit operates as a purely combinational logic block.

The control unit takes instruction fields such as opcode (and sometimes funct3 and funct7) as input and produces control signals that coordinate the register file, ALU, memory, and multiplexers. These signals ensure correct data flow and proper execution of each instruction type.

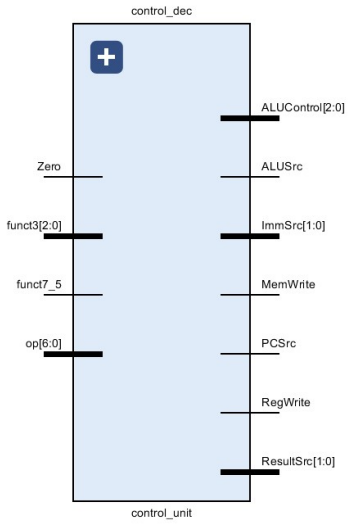

Fig. 7. Control Unit Schematic

The main control signals include *RegWrite*, *ALUSrc*, *MemRead*, *MemWrite*, *MemToReg*, *Branch*, and *ALUOp*. These signals determine whether data is read or written, select ALU inputs, and control memory and register operations.

Different instruction types generate different control settings. For example, R-type instructions enable register writing and ALU operations, load instructions enable memory read and write-back, while store instructions activate memory write without register update. Branch instructions use ALU comparison to decide control flow changes.

The ALU control unit further refines the ALU operation using ALUOp along with funct3 and funct7 fields.

VII. Implementation in Verilog HDL

The proposed single-cycle RISC-V processor is implemented using Verilog HDL in a modular and hierarchical

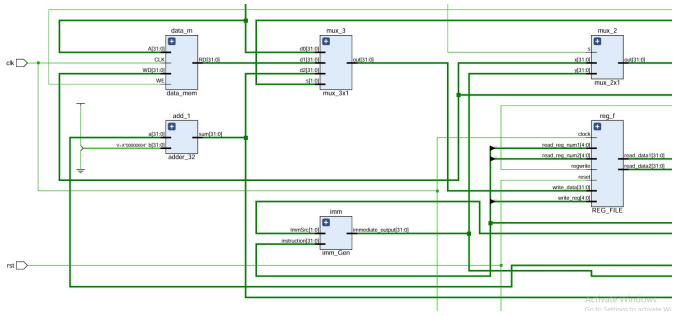

Fig. 8. RTL Schematic Generated by Vivado

design approach. The overall architecture is divided into key functional blocks, including the program counter, instruction memory, register file, arithmetic logic unit (ALU), control unit, data memory, and immediate generator. Each module is designed independently to ensure clarity, reusability, and ease of debugging, and then integrated at the top level to form the complete processor datapath. The control unit is implemented using combinational logic that decodes the instruction opcode and generates the required control signals, while the ALU performs arithmetic and logical operations based on control inputs. The register file supports simultaneous reading of two source registers and writing to a destination register, and memory modules are modeled using behavioral constructs suitable for simulation. This modular Verilog design ensures correct instruction execution within a single clock cycle while maintaining a clear and scalable structure.

VIII. Simulation Results

The proposed single-cycle RISC-V processor is implemented and verified using Vivado 2025 design suite. The target hardware platform is the Xilinx Artix-7 FPGA, specifically the Nexys A7 board (CSG324-100T-1 speed grade). The design is synthesized and implemented using Vivado's synthesis and implementation tools, followed by bitstream generation for FPGA programming. The use of the Artix-7 FPGA provides a reliable and widely used reconfigurable platform for validating digital processor architectures.

TABLE I
Control Signals for RISC-V Type Instruction (R and I Type)

Instr	Opcode	F3	F7	ImmSrc	RegWrite	ALUSrc	ALUOp	MemWrite	ResultSrc	Branch
add	0110011	xx	xx	xx	1	0	000	0	0	0
lw	0000011	010	-	00	1	1	010	0	1	0

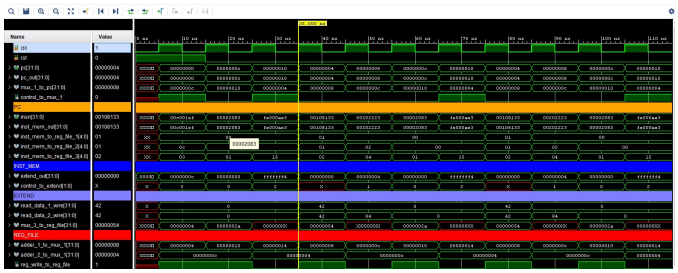

Fig. 9. Execution of R-Type and I-Type Instructions

TABLE II
CONTROL SIGNALS FOR RISC-V TYPE INSTRUCTION (S AND B TYPE)

Instr	Opcode	F3	F7	ImmSrc	RegWrite	ALUSrc	ALUOp	MemWrite	ResultSrc	Branch
sw	0100011	010	-	01	0	1	010	1	x	0
beq	1100011	001	-	10	0	0	110	0	x	1

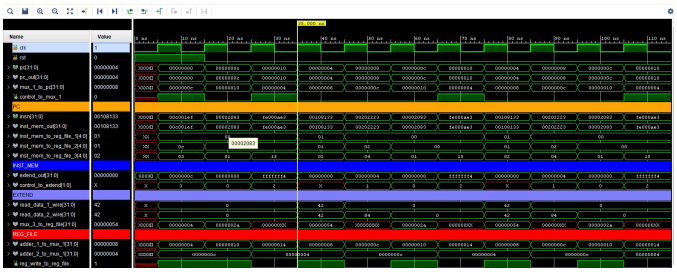

Fig. 10. Execution of S-Type and B-Type Instructions

TABLE III
CONTROL SIGNALS FOR RISC-V TYPE INSTRUCTION (J TYPE)

Instr	Opcode	F3	F7	ImmSrc	RegWrite	ALUSrc	ALUOp	MemWrite	ResultSrc	Branch
jal	1101111	xx	11	1	x	1	xxx	0	1	0

Fig. 11. Execution of J-Type Instructions

IX. RESULTS AND DISCUSSION

Simulation results verified correct processor operation. The register file, ALU, memory subsystem, branch logic, and jump mechanism behaved according to the RV32I specification. All supported instruction formats completed execution within a single clock cycle.

X. CONCLUSION

A 32-bit Single-Cycle RISC-V Processor was successfully designed and verified in Vivado. The processor correctly executed R-Type, I-Type, S-Type, B-Type, and J-Type instructions. The project demonstrates the fundamental operation of RISC-V architecture and serves as a basis for future pipelined processor implementations.

REFERENCES

[1] The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA.
[2] D. Patterson and J. Hennessy, *Computer Organization and Design: RISC-V Edition*, Morgan Kaufmann.
[3] Xilinx, *Vivado Design Suite User Guide*.